Movie Genie

Software Requirements Specification

Version 1

02/19/2026

Group 3
Ajay Rao, Daniel Bojorquez, William Barga

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2023

Revision History

Date	Description	Author	Comments
2-19-2026	Version 1	Ajay Rao, Daniel Bojorquez, William Barga	First draft/revision of SRS
3-5-2026	Version 2	Ajay Rao, Daniel Bojorquez, William Barga	Added UML/SWA diagrams and explanations.

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Ajay Rao, Daniel Bojorquez, William Barga	Software Eng.	Feb 19, 2026
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces *3*
3.1.2 Hardware Interfaces *3*
3.1.3 Software Interfaces *3*
3.1.4 Communications Interfaces *3*
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> *3*
3.2.2 <Functional Requirement or Feature #2> *3*
3.3 Use Cases 3
3.3.1 Use Case #1 ..... *3*
3.3.2 Use Case #2 ..... *3*
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> *3*
3.4.2 <Class / Object #2> *3*
3.5 Non-Functional Requirements 4
3.5.1 Performance *4*
3.5.2 Reliability *4*
3.5.3 Availability *4*
3.5.4 Security *4*
3.5.5 Maintainability *4*
3.5.6 Portability *4*
3.6 Inverse Requirements 4
3.7 Design Constraints 4
3.8 Logical Database Requirements 4
3.9 Other Requirements 4

4. Analysis Models 4

4.1 Sequence Diagrams 5
4.3 Data Flow Diagrams (DFD) 5
4.2 State-Transition Diagrams (STD) 5

5. Change Management Process 5

A. Appendices 5

A.1 Appendix 1 5
A.2 Appendix 2 5

1. Introduction

1.1 Purpose

The purpose of this document is to clarify and explain the purpose, requirements, design, functionality, and constraints of Movie Genie. The intended audience of this document includes the developers of the project, the clients, and Professor Gus Hanna.

1.2 Scope

The finished product will be titled Movie Genie. It will be an online website where users can make an account and purchase memberships or tickets to movie screenings at local theaters. The program should be accessible online and in person via kiosks, allowing maximum convenience for the user. The product will also feature an administrator mode, which will allow for the monitoring of daily ticket purchases. The site will also scrape feature movie reviews and critic quotes from other online sources. The goal of this new product is to make the process of purchasing tickets easier for customers, as well as eliminate the need to maintain the previous system with updates and bugfixes.

1.3 Definitions, Acronyms, and Abbreviations

1.3.1 Client: The organization/individuals for which this product is being developed.
1.3.2 User: Any individual who directly interacts with and uses the Movie Genie software.
1.3.3 Customer: Any user (as defined above) who uses Movie Genie for the purposes of browsing movie showtimes, making a purchase, or changing their account status/information.
1.3.4 AMC: Refers to AMC Entertainment Holdings, Inc, the business entity for which this product is being developed.
1.3.5 API: Application Programming Interface: a set of protocols that allow two separate programs to communicate data with one another.
1.3.6 NFT: Non-Fungible Token: unit of data that verifies the uniqueness and ownership of a digital asset.

1.4 References

IEEE Recommended Practice for Software Requirements Specifications, 1998, IEEE Computer Society, Software Engineering Standards Committee

1.5 Reflection

Creating this Software Requirements Specification helped our team better understand the complexity of building a modern ticketing system. While purchasing tickets seems simple, documenting the requirements revealed the need for secure payments, anti-bot protection, real-time seat updates, third-party integrations, and high system reliability.

This process emphasized the importance of clearly defining functional and non-functional requirements, system constraints, and external dependencies early in development. Overall, this

SRS provides a clear foundation for implementing Movie Genie in a structured and organized manner.

2. General Description

2.1 Product Perspective

Movie Genie is a new software meant to replace the aging system that was already in place. It will be a web-based system that connects to the database of the local theater chain. It will utilize third-party payment systems, identity verification services, scraping of reviews from popular review sites, and the AMC Membership database.

2.2 Product Functions

Here are the main functions that the product will perform:

2.2.1 It will allow customers to purchase tickets and select seats to movie screenings.
2.2.2 Track purchase history as well as loyalty/membership accounts and points.
2.2.3 Anti-ticket-scalping measures, unique ticket generation, and login limitations to one device at a time.
2.2.4 Administrator mode that allows live monitoring of ticket sales and customer feedback.
2.2.5 Features movie reviews scraped from popular review sites for customers to view.

2.3 User Characteristics

The users of this product are expected to have a large range in technological literacy. Users could range from tech-savvy customers using Movie Genie from their mobile browser, to more average customers using the in-person kiosks. The advantage of the kiosks is that there will be nearby employees to help explain the process. Any administrative users are expected to be familiar with the product and how it works, either through training or reading this SRS.

2.4 General Constraints

The general limitations are as follows:

2.4.1 The budget is currently limited to $500k.
2.4.2 Deadline of 4 months for a viable prototype.
2.4.3 Should operate strictly on the Pacific Time Zone with the San Diego theater chain.
2.4.4 Must run on standard popular browsers, as well as theater kiosk hardware.

2.5 Assumptions and Dependencies

The product shall be dependent on ID.me and AMC APIs for verifying discounts and memberships. The software will also be dependent on the user and theaters having stable connection to the internet and local networks respectively.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

3.1.1.1 Web Interface:

- Accessible on popular web browsers such as Chrome, Safari, etc.
- Has language support for English, Spanish, and Swedish.
- Have a header, footer, and menu that are accessible at all times
- Seat maps that are color coded to show which seats are available for deluxe theaters
- Ticket purchase timer that is 5 minutes long
- Smiley face feedback prompt

Priority: Very high, necessary for smooth customer experience

3.1.1.2 Administrator Interface:

- Logs for daily ticket sales
- The amount of tickets that are sold per movie
- Data on pricing and revenue
- Management tools for showtimes and the theater (in the event of technical difficulties)
- Queue monitoring

Priority: Very high, necessary for adequate monitoring of sales information

3.1.2 Hardware Interfaces

1) Kiosk touchscreen / display - Touchscreen operated device. Can operate off of a custom operating system or launcher, as long as it can run a web browser.
   - Priority: Very High, necessary for kiosk functionality.
2) Network Interface - Stable internet or local network connection.
   - Device - Network adapter in kiosk or physical cable.
   - Interface - TCP/IP over ethernet/wifi
     Priority: Very High, necessary for communication with databases and servers.
3) Ticket Printer - Used for printing physical tickets.
   - Device - Ticket printer
   - Interface - USB network
     Priority: High, necessary for physical tickets but not necessary for product functionality.
4) QR Code - Used to scan and validate tickets.
   - Device - ID/2D Barcode Scanner, Company tablet with a camera
   - Interface - Usually acts like a keyboard or via serial/ USB SDK.
     Priority: Very High, essential to verifying ticket validity

3.1.3 Software Interfaces

1) Web Browser Interface - Users access Movie Genie through this browser. Interface can be HTML, CSS or javascript.

2) Payment(API):
   a) Third party credit card processing service(eg Applepay, etc)

   b) PayPal
   c) Bitcoin Wallet
3) Theater Database System - Will use the local theater ticket inventory system. The interface is usually API.
4) Database Management System - Uses SQL queries to store User accounts, Membership details, ticket purchases etc.

3.1.4 Communications Interfaces

Should have HTTPS for all communication on the web, encrypted API calls to payments, polling to update seat availability, and use Pacific Time for all timestamps.

3.2 Functional Requirements

This section contains the major functional features, broken down into inputs, processing, outputs, and error handling.

3.2.1 Ticket Purchasing and Reservation

3.2.1.1 Introduction
This feature lets customers purchase tickets for a showtime of their choice. If the customer chooses a deluxe theater, they must select a seat.
3.2.1.2 Inputs
Users must be able to select the date, theater, or movie, then select which showing. They should also be able to select seats, amount of tickets, discount if available, and a payment method.
3.2.1.3 Processing
First, the system gets showtimes from the database, then displays the seat map for the deluxe theaters, then checks to see if the seats are available and if there are any eligible discounts. It should start the timer for the process, make sure the user is purchasing a maximum of 20 tickets, and then reserve the seats chosen for those 5 minutes. Then it must process the payment, generate the NFT tickets, and send them to the user (either digitally or physically). Finally it must log the transaction.
3.2.1.4 Outputs
The system lists all movies and showtimes, then the seat map if available, then ticket prices and available discounts. After the purchase is complete, the user must get a confirmation receipt and the ticket must be delivered to them.
3.2.1.5 Error Handling
If the showtime is sold out then display “Sold Out”. If the showtime isn’t during the purchase window then display “Unavailable”. If the payment fails, then display “Payment Failed”. If the timer runs out, display “Time has expired”

3.2.2 Loyalty Account Management

3.2.2.1 Introduction
This feature lets users create loyalty accounts that store personal data, payment methods, purchase history, and loyalty.
3.2.2.2 Inputs

The user must input their name, email, and password. They will have the option to enter their payment information and AMC membership information and save them.
3.2.2.3 Processing
The account will first be created or signed into, then make sure that they are using their device and not someone else's. Then their payment and personal information will be encrypted and stored. Purchase history and accumulated loyalty points will be tracked. When processing an exchange, the eligibility to exchange must be validated by making sure that it was the same account that purchased the original ticket.
3.2.2.4 Outputs
There will be current loyalty points balance and purchase history displayed on a dashboard for the account. In the circumstance of an exchange, a confirmation message will appear.
3.2.2.5 Error Handling
If the login credentials are invalid, display "Invalid email or password". If the account is logged into another device, display "Account already active". If the user attempts to exchange the ticket after the showtime, display "Cannot exchange post-showtime". If the account is created with an email that is already being used, display "Account already exists".

3.2.3 Queuing System

3.2.3.1 Introduction
This feature makes it so users have fair access to buying a ticket when showtimes are in high demand.
3.2.3.2 Inputs
The user first requests tickets for a high demand showtime.
3.2.3.3 Processing
The users are placed into a standard FIFO queue. Creates a timer when they leave the queue to buy tickets.
3.2.3.4 Outputs
The position in the queue will be displayed, as well as a notification when the user leaves the queue. Also a notification will appear if the user runs out of time to buy the tickets.
3.2.3.5 Error Handling
If a user disconnects, they will be removed from the queue. If the queue time expires, the user's session will be canceled.

3.2.4 Anti-Bot Protection

3.2.4.1 Introduction
This feature prevents bots from purchasing large amounts of tickets.
3.2.4.2 Inputs
The user's session behavior, IP address, and CAPTCHA response will be input.
3.2.4.3 Processing
CAPTCHA verification will be required and flagged IP addresses will be blocked. Purchasing patterns will be monitored and checked for abnormalities.
3.2.4.4 Outputs

The user will be prompted with a CAPTCHA challenge, and whether they successfully do it or not will give them either an access grant or denied message.
3.2.4.5 Error Handling
If the user fails the CAPTCHA , the system will prompt the user again. If they fail it multiple times, the IP will be blocked temporarily.

3.2.5 Administrator Mode

3.2.5.1 Introduction
This feature allows administrators to manage capabilities of the theaters, such as showings, pricings, and reports.
3.2.5.2 Inputs
Administrators must login with their credentials. They can input their reports as well.
3.2.5.3 Processing
First the system must verify that the account belongs to an administrator. The system must retrieve daily logs and ticket sales data.
3.2.5.4 Outputs
There must be reports of the revenue and a sales dashboard, along with downloadable reports.
3.2.5.5 Error Handling
If the account trying to access the administrator mode isn't authorized, deny access.

3.2.6 Discount Handling

3.2.6.1 Introduction
This feature applies discounts to purchases if eligible.
3.2.6.2 Inputs
The user selects a movie they'd like to watch, then selects their desired discount ticket option. Then, the user verifies their eligibility for the discount. For the military, they will sign in using ID.me, for students they will sign in through their school account, and seniors will have their ID checked in person. For the AMC members, they must sign in through AMC membership.
3.2.6.3 Processing
First the system will validate the eligibility for the discount, then verify that the day of the ticket matches that eligible for the discount, as students and military only get discounts on weekdays. Then the price reduction will be applied, and the total cost will be updated.
3.2.6.4 Outputs
The system will show the updated ticket price and confirm the discount was applied.
3.2.6.5 Error Handling
If the discount selected isn't validated, display "Validation Failed". If a weekday discount is attempted to be applied to a weekend showtime, display "Discount may only be applied to weekday showtimes".

3.2.2 Movie Review Integration

3.2.7.1 Introduction
This feature displays critic ratings and reviews from certain review sites.
3.2.7.2 Inputs

The system requires the movie title and review data from external sources.
3.2.7.3 Processing
First the review data must be obtained, then displayed on the website.
3.2.7.4 Outputs
The ratings and review quotes will be displayed.
3.2.7.5 Error Handling
If there are no reviews available, display “Reviews unavailable”.

3.2.8 Customer Feedback System

3.2.8.1 Introduction
This feature collects feedback from the customer.
3.2.8.2 Inputs
The user will select a face and write down any other comments they would like to share.
3.2.8.3 Processing
The feedback prompt will be displayed first. Then the feedback will be stored with the transaction ID.
3.2.8.4 Outputs
The smiley face feedback prompt will be displayed, a thank you confirmation message will be displayed after submitting the feedback. A report will be displayed to administrators.
3.2.8.5 Error Handling
If submission fails, display “Submission failed, please try again” and prompt the user again.

3.3 Use Cases

3.3.1 Use Case #1

Name of Use Case: Purchase Tickets Online
Actor: Customer
Flow of events:

1. The customer will browse and select which movie they want to watch.
2. The customer selects one of the available showtimes to purchase tickets for.
3. The movie selected is in a deluxe theater, so the customer chooses their preferred seats. Available seats will be shown in
4. The user selects the number of tickets they want to purchase For a deluxe theater, the number of tickets must match the number of selected seats from the previous step in order to proceed.
5. If the customer selects military or student discount tickets, the system verifies that it is a weekday so discounts are valid.
6. Customer is redirected to ID.me or their school portal to log in and verify their eligibility. For senior tickets, their age will be verified with ID at the theater in person.
7. The customer will be prompted to complete a CAPTCHA to verify they are human.
8. The system will calculate and display order details and checkout total.
9. The customer will input their payment information and email, then confirm the purchase.

10. The customer will receive a copy of their ticket ID and QR code in their email, as well as a receipt of their purchase.

3.3.2 Use Case #2

Name of Use Case: Administrator View Sales
Actor: Admin
Flow of events:

1. Admin logs into the administrator dashboard using verified account information.
2. Admin selects a specific movie or theater to view reports.
3. The system retrieves live-updated data on capacity, tickets sold, and revenue.
4. Admin can filter the displayed information by date or discount type.
5. System displays filtered or unfiltered information from step 3, and also displays customer feedback reports which can be sorted through manually.

3.3.3 Use Case #3

Name of use Case: Customer Purchases Membership
Actor: Customer
Flow of events:

1. The customer selects the “purchase membership” button/link.
2. The customer is prompted to log into or create an account if not already signed in.
3. The customer selects which tier of membership they would like to subscribe to.
4. The customer reads and verifies they have read the terms and conditions, provided by AMC, of the membership.
5. The system will calculate and display order details and checkout total.
6. The customer will input their payment information and email, then confirm the purchase.
7. The customer will receive a receipt of their purchase in their email, and their account is upgraded to whatever membership they selected.

3.4 Classes / Objects

3.4.1 Auditorium

3.4.1.1 Attributes:
TheaterNumber, Tier (normal or deluxe), MaxSeats, CurrentCapacity, SeatList (2D array of seat id’s for deluxe theaters)
3.4.1.2 Functions:

- checkAvailability(): returns number of available seats
- reserveSeat(): reserves a seat and subtracts from CurrentCapacity

3.4.2 Movie

3.4.2.1 Attributes:
Title, Genre, Rating, Runtime, ReviewScore, Description, Image
3.4.2.2 Functions:

- getShowtimes(): returns a list of movie showtimes from the database
- updateReviews(): checks review site API for new reviews to scrape for the movie

3.4.3 Ticket

3.4.3.1 Attributes:
TicketID, SeatNumber (deluxe theaters), Price, TokenID, TimeStamp, Validity
3.4.3.2 Functions:

- generateQRCode(): generates a unique qr code for the ticket
- checkValidity(): checks if the ticket has been used/scanned yet, or if showtime has passed
- markAsUsed(): turns Validity to false, rendering ticket invalid

3.4.4 Account

3.4.4.1 Attributes:
Username, Password, AccountID, UserEmail, UserIP, MembershipTier, Purchases, Points
3.4.4.2 Functions:

- verifyLogin(): verifies username and password are correct when logging in
- storePurchase(): sends receipt of purchases to users email and stores it in Purchases
- addPoints(): adds points to Points after user completes purchase or redeems awards
- changeMembershipTier(): upgrades or downgrades membership tier after membership purchase or cancellation

3.4.5 Showtime

3.4.5.1 Attributes:
ShowtimeID, MovieID, AuditoriumID, StartTime, TicketPrice, AvailableSeats
3.4.5.2 Functions:

- getAvailableSeats(): Returns all the seats that are currently available
- isWithinPurchaseWindow(): Returns true is the showtime is within the purchase window
- reserveSates(): attempts to reserve specified seats and returns true if they are successfully reserved.

3.4.6 Payment

3.4.6.1 Attributes:
PaymentID, AccountID, Amount, Method, Timestamp, ReferenceCode
3.4.6.2 Functions:

- processPayment(): Sends a payment request to the external gateway and returns true if the payment is approved
- handlePaymentResponse(): Interprets the gateway's response and determines whether it is a success or failure
- refundInPerson(): Marks the payment as refunded

3.4.7 Discount

3.4.7.1 Attributes:
DiscountID, Type, PercentOff, WeekdayOnly, VerificationService
3.4.7.2 Functions:

- verifyEligibility(): Checks to see if the user is qualified for the discount
- applyDiscount(): Returns the discounted price
- requestExternalVerification(): Sends verification request to the external service if required

3.4.8 Theater

3.4.8.1 Attributes:
TheaterID, Name, Location, Type, SeatCapacity, Auditoriums
3.4.8.2 Functions:

- getAuditorium(): Returns all of the auditoriums in the theater
- getShowtimes(): Returns all of the showtimes scheduled at this theater

3.5 Non-Functional Requirements

3.5.1 Performance

The website should be able to handle up to 1,000 simultaneous users, and be scalable up to 10 million concurrent users. 95% of transactions should take no longer than three seconds. Under high load/demand, the website will begin using a queue to handle requests.

3.5.2 Reliability

The website will keep daily logs of purchases and transactions. There should be a data backup every 24 hours, and the MTBF should be upwards of 30 days.

3.5.3 Availability

The website should be up and available 99% of each month. Data and information should update live and be consistent across websites and kiosks.

3.5.4 Security

- Each ticket should be an NFT, that is unique and non-replicable. System will use a unique identifier generator to ensure this.
- Payment information should be encrypted when processed.
- Concurrent user logins should be limited to a single device at a time.
- Anti-scalping mechanisms should prevent bots mass purchasing tickets.
- Will use secure HTTPS communication with servers.

3.5.5 Maintainability

Should use a modular architecture and partitions the main database into different sections/theaters.

3.5.6 Portability

Will be programmed using HTML for website formatting and Java for object handling, as these languages work across all major operating systems. The product should be compatible with most modern web browsers such as Chrome or Safari, with no dependency on a mobile app.

4. System Overview

The Movie Genie System is a browser based ticketing system for a chain of 20 San Diego theaters. It supports both online and in-person kiosk ticket purchases, general admission for regular theaters, assigned seating for deluxe theaters, loyalty accounts, AMC memberships, NFT-based tickets, administrative reporting, and multi-language support. The system uses a centralized database that is split by theater. It must support anti-bot protection, high concurrency, and seat availability that is updated in real time.

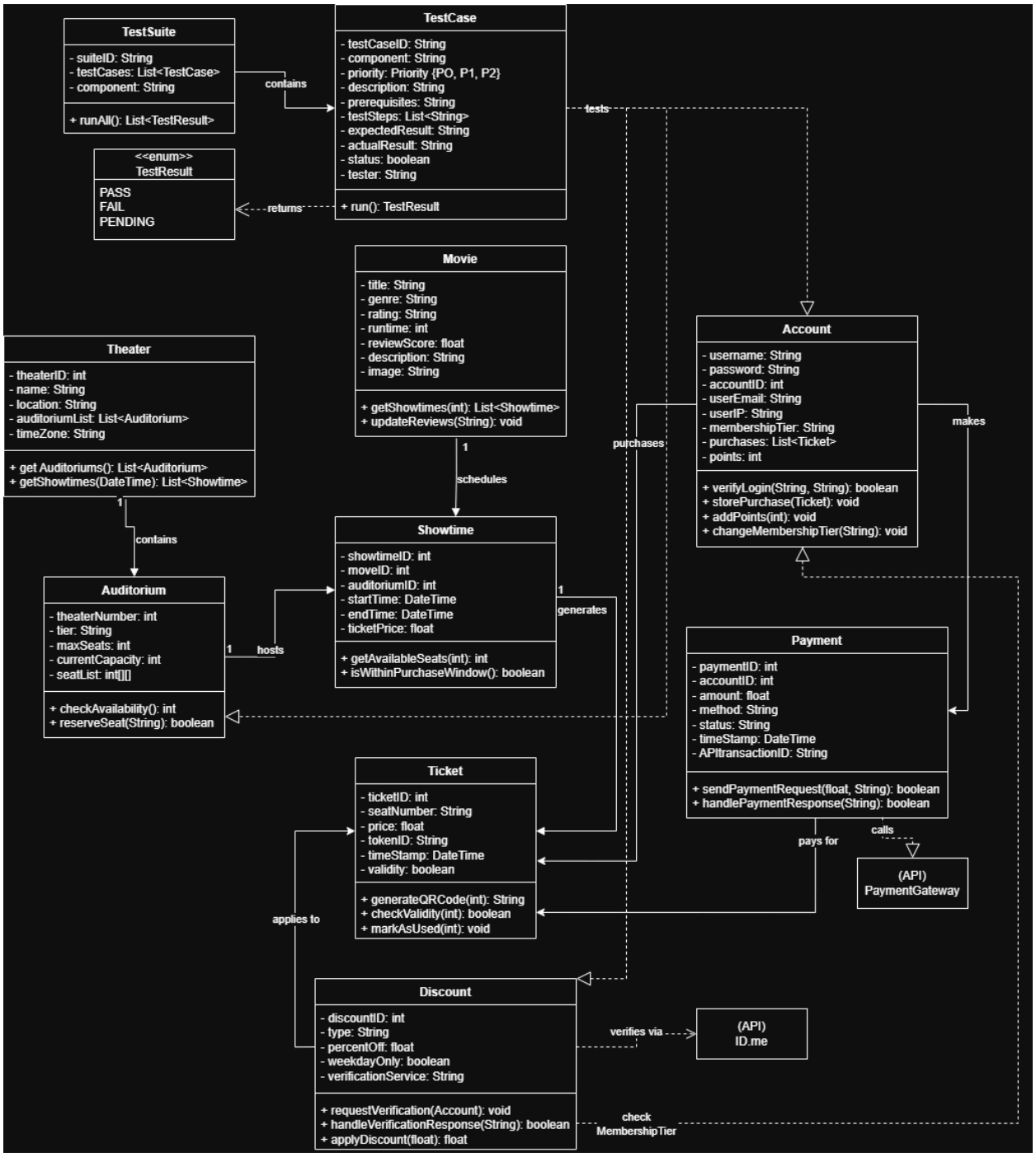
5. Software Architecture Overview

5.1 Architectural Overview

The system follows a layered, modular architecture:

- Presentation Layer; Kiosk UI, Web UI, Admin Dashboard
- Application Layer; Business Logic, Queue Manager, Anti-Bot Module, Discount Engine
- Integration Layer; NFT Blockchain API, AMC Membership API, Payment Gateway API, Review Scraper API
- Data Layer; Theater Partitions, Ticket Record, Central Database, User Accounts, Logs

5.2 UML Diagram



5.3 UML Explanation

This section provides an overview of the UML Class Diagram for Movie Genie. Each class is described in terms of its attributes, operations, purpose, and relations to other classes. Together, these classes define the important data structures and behaviors that support ticket purchasing, user accounts, showtime management, payment processing, and discounts.

5.3.1 Theater Class

The Theater class represents the location within the San Diego theater chain. Each theater contains multiple auditoriums and operates at Pacific Standard Time.

5.3.1.1 Attributes:

- theaterID: int
    - Unique identifier for the theater
- name: String
    - Name of the theater
- location: String
    - Physical address
- auditoriumList: List<Auditorium>
    - Collection of auditoriums for each theater
- timeZone: String
    - Time zone for scheduling (PST)

5.3.1.2 Operations:

- getAuditoriums(): List<Auditorium>
    - Returns all auditoriums in a certain theater
- getShowtimes(DateTime): List<Showtime>
    - Retrieves all showtime on a certain date.

5.3.1.3 Relationships:

- An Theater contains multiple Auditoriums

5.3.2 Auditorium Class

The Auditorium class represents an individual room in the theater. It tracks capacity and availability for seating.

5.3.2.1 Attributes:

- theaterNumber: int
    - Identifier for an auditorium within the theater
- tier: String
    - Shows whether the auditorium is Regular or Deluxe
- maxSeat: int
    - Total number of seats
- currentCapacity: int
    - The number of seats that are currently available
- seatList: int[]
    - Array representing the seat identifiers

5.3.2.2 Operations:

- checkAvailability(): int
    - Returns the number of seats that are available
- reserveSeat(String): boolean
    - Tries to reserve a seat; returns true if its successful

5.3.2.3 Relationships:

- An Auditorium hosts multiple Showtimes

5.3.3 Movie Class

The Movie class stores data about each film shown in the San Diego theater chain.

5.3.3.1 Attributes:

- title: String
    - Title of the movie
- genre: String
    - Genre classification
- rating: String
    - MPAA rating (PG, PG-13, etc.)
- reviewScore: float
    - Aggregated audience or critic score
- director: String
    - Name of the director
- image: String
    - Thumbnail image URL

5.3.3.2Operations:

- getShowtimes(int): List<Showtime>
    - Retrieves showtimes for this movie at a certain theater
- updateReview(String): void
    - Updates the critic quote or review score

5.3.3.3 Relationships:

- A Movie is associated with multiple Showtimes

5.3.4 Showtime Class

The Showtime class represents a screening of a movie in a specific auditorium at a certain time.

5.3.4.1 Attributes:

- showtimeID: int
    - Unique identifier
- movieID: int
    - ID of the movie being shown
- auditoriumID: int
    - ID of the auditorium hosting the movie
- timeSlot: DateTime
    - Date and time of the showing
- ticketPrice: float
    - Base price of ticket before discounts

5.3.4.2 Operations:

- getAvailableSeatList(): int[]
    - Returns a list of unreserved seats
- isWithinPurchaseWindow(): boolean
    - Checks if the showtime is within the purchase window

5.3.4.3 Relationships:

- A Showtime belongs to a Movie and Auditorium

- A Showtime is referenced by Tickets

5.3.5 Ticket Class

The Ticket class represents a reserved or purchased ticket, including NFT-based authentication.

5.3.5.1 Attributes:

- ticketID: int
    - Unique ticket identifier
- seatNumber: String
    - Seat assigned
- price: float
    - Final price after any discounts
- tokenID: String
    - NFT token identifier
- timestamp: DateTime
    - Time of purchase
- validity: boolean
    - Indicates whether or not the ticket is valid

5.3.5.2 Operations:

- generateQRCode(): String
    - Produces a QR code that can be scanned
- checkValidity(): boolean
    - Verifies whether the ticket is still valid or not
- markAsUsed(): void
    - Marks the ticket as redeemed

5.3.5.3 Relationships:

- A Ticket belongs to a Showtime
- A Ticket may belong to an Account

5.3.6 Discount Class

The Discount class handles discount logic for students, seniors, military, and AMC members.

5.3.6.1 Attributes:

- discountID: int
    - Unique identifier
- type: String
    - Discount category
- percentOff: float
    - Percentage discount
- isMajorCity: boolean
    - Indicates whether discount rules vary by location
- verificationService: String
    - External service used for verification

5.3.6.2 Operations:

- requestVerification(Account): void
    - Sends verification request to the external service

- handleVerificationResponse(String): boolean
    - Processes the verification response
- applyDiscount(float): float
    - Applies discount to a base price

5.3.6.3 Relationships:

- Discount interacts with ID.me for verification
- Discount may require data from Account

5.3.7 Account Class

The Account class represents a user with optional membership and loyalty features.

5.3.7.1 Attributes:

- username: String
    - Login name
- password: String
    - Hashed password
- accountID: int
    - Unique identifier
- fullName: String
    - User's name
- emailID: String
    - Email address
- membershipTier: String
    - AMC membership tier
- purchases: List<Ticket>
    - Tickets purchased by the user

5.3.7.2 Operations:

- verifyLogin(String, String): boolean
    - Validates login credentials
- purchaseTicket(Ticket): void
    - Adds a ticket to the user's purchase history
- addFunds(float): void
    - Adds funds to the user's account if available
- changeMembershipTier(String): void
    - Updates the AMC membership tier

5.3.7.3 Relationships:

- An Account purchases Tickets
- Account interacts with Discount for verification

5.3.8 Payment Class

The Payment class handles all transactions for purchases.

5.3.8.1 Attributes:

- paymentID: int
    - Unique identifier
- accountID: int

        - ID of the paying account
    - amount: float
        - Total amount charged
    - method: String
        - Payment method
    - Timestamp: DateTime
        - Time of transaction
    - refTransactionID: String
        - Reference ID from payment processor

5.3.8.2 Operations:

- sendPaymentRequest(float, String): boolean
    - Sends a payment request to the external gateway
- handlePaymentResponse(String): boolean
    - Processes approval or denial of payment

5.3.8.3 Relationships:

- Payment interacts with PaymentGateway
- Payment is associated with Ticket purchases

5.3.9 External API Classes

PaymentGateway: Represents the external payment processor that is used by the payment class to authorize transactions.

ID.me: External identity verification service used by Discount to verify military and student status.

5.3.10 TestCase Class

The TestCase class represents a testing scenario used to verify that a certain system component is working correctly. They capture inputs, execution steps, expected and real outcomes, and determine a pass/fail result.

5.3.10.1 Attributes:

- testCaseID: String
    - Unique identifier for the test case
- component: String
    - The system component or module being tested
- priority: Priority
    - Execution priority level, represented as enum: P0, P1, P2
- description: String
    - A summary of what the test case is verifying
- prerequisites: String
    - Any conditioner that must be met before the test can be executed
- testSteps: List<String>
    - Ordered list of actions to perform during the test
- expectedResult: String
    - The anticipated outcome if the system works properly
- actualResult: String

        - The recorded outcome after the test has been executed

5.3.10.2 Operations:

- run(): TestResult
    - executes the test case and compares the actual result to the expected result, returning PASS, FAIL, or PENDING

5.3.10.3 Relationships:

- A TestCase is grouped into one TestSuite
- A TestCase targets one existing system component via a dependency

5.3.11 TestSuite Class

The TestSuite class has a collection of TestCase objects under a single component. It allows grouped execution of associated test cases.

5.3.11.1 Attributes:

- suiteID: String
    - unique identifier for the test suite
- component: String
    - The system component or feature area this suite covers
- testCases: List<TestCase>
    - collection of all TestCase objects belonging to this specific suite

5.3.11.2 Operations:

- runAll(): List<TestResult>
    - executes every test case in the suite and returns list of results

5.3.11.3 Relationships:

- A TestSuite contains one or more TestCases

5.3.12 TestResult Enum

The TestResult enum are the potential outcomes of TestCase executions.

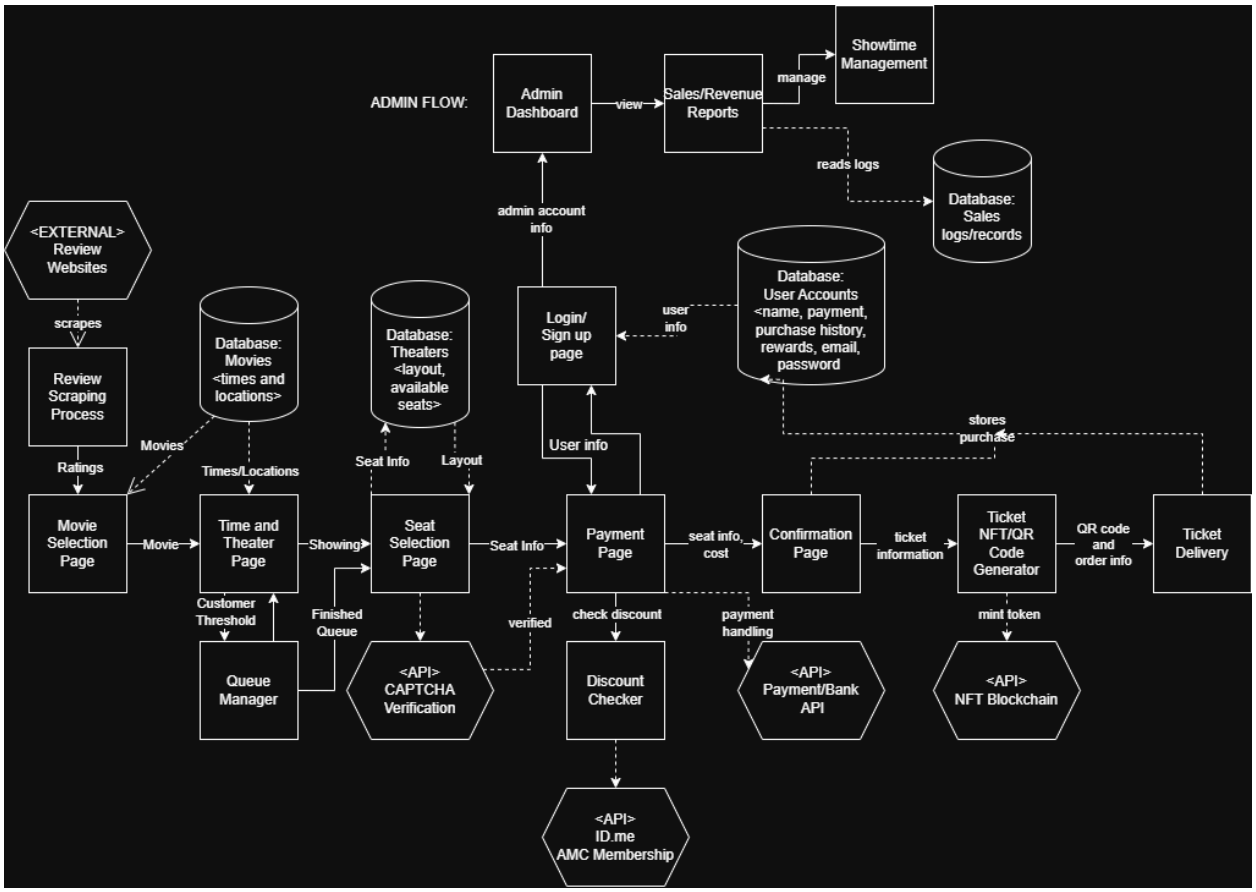
5.3.12.1 Values:

- PASS
    - The actual result matched the expected result
- FAIL
    - The actual result did not match the expected result
- PENDING
    - The test case has not yet been executed (null)

5.3.12.2 Relationships:

- returned by TestCase.run()
- Collected and returned as a list by TestSuite.runAll()

5.4 SWA Diagram



5.5 SWA Explanation

The Software Workflow Architecture diagram shows the complete end-to-end flow of user interactions data, external integrations and backend processes within the system. It shows how administrators, databases, customers, APIs, and internal services interact to support set selection, payment, ticket generation, administrative operations, and movie browsing. The architecture is split into three flows: Customer Flow, Admin Flow, and External Integration Flow.

5.5.1 External Review and Movie Browsing Flow

This workflow starts with the integration of the system with external review websites, which provides ratings from critics, review scores, and summary quotes. A review scraping process periodically collects this information and updates the system's movie records. These ratings are then displayed on the Movie Selection Page, making sure that users can see accurate and current reviews while browsing films. The Movie Selection Page retrieves movie titles, ratings, genres, and showtime availability from the Movies database. When a user selects a film, the system loads all showtimes, theater locations, and dates onto the Theater and Time pages.

5.5.2 Queue Management and Seat Selection Flow

After selecting a showtime, the user may be routed through the Queue Manager if the showing is in high demand. It regulates traffic by assigning queue positions, enforcing customer thresholds, and then releasing users into the next stage when it's able. Once the user reaches the Seat Selection Page, the system gathers the layout of the theater, seat map, and availability from the Theaters database. This makes sure that the users can see accurate seat availability. After the user selects their seats, the system forwards all the data to the Payment Page.

5.5.3 Login, Discount Verification, and Payment Flow

Before completing a payment, users will get the option to log in or create an account through the Login Page. This page communicates with the User Accounts database to gather payment methods, loyalty points, membership tier, and purchase history. Once the user proceeds to payment, the system checks to see if they are eligible for any discounts by sending the user's data to the Discount Checker. If verification is needed, Discount Checker communicates with the necessary external services. After discounts are applied, the Payment Page sends the final payment request to the external Payment or Bank API, which returns either an approval or denial with a transaction ID. Successful transactions are stored in the User Accounts database, loyalty points are then updated, and the user is directed to the Confirmation Page.

5.5.4 Ticket Generation and Delivery Flow

Once a payment is confirmed, the system generates the user's ticket. The Confirmation Page sends ticket data to the Ticket NFT or QR Code Generator, which creates both a QR code and NFT token ID. The NFT service communicates with the NFT Blockchain API to make a blockchain-verified token, making sure that the ticket is unique, and sends the user the NFT token, QR code, and order summary through an email or onscreen download.

5.5.5 Administrative Flow

The Software Workflow Architecture diagram also has the administrative workflow, which can be accessed through the Admin Dashboard. This dashboard gathers purchase logs, system activity, and account information from the User Accounts and Sales Logs databases. Administrators can see sales and revenue reports, monitor system health, and analyze ticket performance. Also, the Showtime Management module lets administrators remove, add, or modify screenings, prices, and theater availability.

6. Development Plan and Timeline

6.1 Partitioning of Tasks/Responsibilities

Partitioning of tasks will largely be dynamic and dependent on the specific task to be completed. Each team member will be responsible for completing the following requirements in regards to their own designated tasks:

6.1.1 Dev Log: At least a short description of progress/changes they've made on their task.

6.1.2 Bug Log: List of any bugs/errors (new or pre-existing) encountered while trying to complete maintenance tasks.

6.1.3 Time Management: Team members are expected to give time estimates of how long they need to complete certain tasks. If they fail to meet their deadline, they must communicate it with their teammates then reach a solution together.

7. Test Plan

7.1 Unit Testing

7.1.1 AuditoriumAvailability

- Tests Auditorium.checkAvailability() by creating an instance of an Auditorium object with currentCapacity = 75 and maxSeats = 100 with an expected return value of 75. This confirms that the method functions correctly without external dependencies.

7.1.2 AuditoriumReserve

- Tests Auditorium.reserveSeat() by initializing currentCapacity = 50, calling reserveSeat("A5"), and asserting that currentCapacity decreases to 45. This verifies that the method correctly mutates its internal state with a valid reservation input.

7.1.3 SeatSelection_Unavailable

- This test works in tandem with the previous one. If the user attempts to select an occupied seat, the test asserts that confirmation will be blocked and an error will be returned.

7.1.4 Login_InvalidPassword

- Tests Account.verifyLogin() using blackbox testing method. Valid credentials expect access granted, invalid credentials expect error message. The test supplies a valid email with a wrong password, representing invalid value, and asserts the system displays an error without granting access.

7.1.5 Search_ValidQuery

- Tests the search module by submitting "Avengers" as a valid query and expects relevant results to be returned. The input space recognizes valid/unvalid queries (titles vs empty/meaningless text). This test covers valid inputs.

7.2 Integration Testing

7.2.1 DiscountStudent

- Tests the interface between Discount class, ID.me API, and Ticket class pricing. The Discount class calls requestVerification(), ID.me responds with handleVerificationResponse(), and applyDiscount() then applies discount to the ticket price. The test asserts that the correct price (discounted or not) is displayed on the payment page, confirming that all three interfaces function without breaking the system. Both true and false scenarios are covered thanks to handleVerificationResponse().

7.2.2 SeatLock

- Tests the interface between Auditorium and Showtime classes. When seats A1 through A3 are selected, Auditorium.reserveSeat() must communicate with Showtime to make those seats unavailable to all other users for the next five minutes. The test asserts that the lock is correctly communicated with the time window.

7.2.3 Queue_HighDemand

- Tests the interface between the queue manager and the Showtime component under high-load scenarios. The test enters the queue for a high-demand showing and expects a queue position to be correctly assigned/displayed. This would confirm that the two components are properly exchanging state when capacity is limited.

7.2.4 Admin_ViewReports

- Tests the interface between the admin account, dashboard component, and underlying sales data. The test logs in as an admin, navigates to the reports, and asserts that sales data and logs are correctly retrieved and displayed. This would verify that the data flows correctly from storage to the admin view.

7.3 System Testing

7.3.1 SystemPurchase

- Primary system test, covering the full ticket purchase pipeline. The test sequence is as follows:
    1. Select Movie
    2. Select showtime
    3. Select seats
    4. Apply a discount
    5. Complete payment
    6. Receive ticket

This traverses the entire call chain across Account, Showtime, Auditorium, Discount, Payment, and external PaymentGateway and ID.me APIs, and Ticket generation. Expectation is that a successful test ends with a ticket being generated with valid TokenID, which is then delivered to the user. This test validates that no failures should result from regular user interaction with the system.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2